



Reflection

DEPLOYMENT & INSTALLATION GUIDE

PART I · VERSION 1.0



STEALTH
ARCHITECTURE



ZERO
EXPOSURE



ENCRYPTED
BY DESIGN

1 What Reflection does

Reflection publishes a TCP or UDP service that is reachable from a public mirror server, while the real service stays on a private reflector server. A reflector creates encrypted control/data channels to the mirror. When a client connects to the published mirror port, Reflection assigns one channel and connects to the configured origin service.

Client → public mirror:published port → encrypted Reflection channel → reflector → private origin service

Figure 1. Connection path from a client to the private origin service.

This is a transport-layer relay. It does not understand HTTP, TLS, databases, or application protocols; it forwards TCP streams or UDP datagrams. Use it only where a reverse TCP/UDP relay is the intended architecture.

2 Components and where they run

Table 1. Reflection components and where they run.

Component	Run on	Purpose
Mirror	Public/reachable host	Receives reflector control/data channels and opens the client-facing listener on demand.
Reflector	Network that can reach the origin	Connects to the mirror, maintains a TCP channel pool, and opens connections to the origin.
Origin service	Usually private	The existing TCP/UDP service being made reachable. It is not part of this repository.

3 Requirements

- Node.js with the built-in `net`, `dgram`, `crypto`, `cluster`, and `filesystem` modules. The repository has no `package.json` and no third-party Node dependency install step.
- Two hosts with network connectivity from reflector to mirror control/data ports.
- An origin service reachable from the reflector on the configured host and port.
- Permission to listen on the selected ports. Use ports above 1024 unless the service account has the required capability.
- The mirror and reflector must use the same `conf/crypt.json` values. Control traffic and TCP relay control messages use the shared key; the optional AES extension has its own `conf/aes.json` secret.

Check Node before deployment:

```
node --version
node -e "require('net'); require('dgram'); require('crypto'); console.log('Node runtime OK')"
```

4 Before installation: choose ports and firewall rules

Choose non-conflicting ports. Do not expose the control, heartbeat, or firewall-control ports to the Internet. Restrict them to reflector/admin addresses. The published service port is the only port normally open to clients.

IMPORTANT · UDP

The implementation starts its heartbeat server with Node's TCP `net` API even for the UDP mode. Keep the heartbeat port reachable over TCP until a future release.

5 Installation

Perform these steps on both hosts.

1. Copy the complete reflection directory to a stable location, for example `/opt/reflection`.
2. Create a non-login service account, for example `reflection`.
3. Give that account ownership of the installation and ensure it can create `/opt/reflection/logs`.
4. Keep `conf/` readable only by the service account because it contains secrets.

6 Secure the configuration first

The supplied configuration contains sample/default secrets and example addresses. Treat it as a template, not production configuration.

1. Generate two new high-entropy secrets: one for `key`, one for `heartbeat_key`.
2. Put them in `conf/crypt.json` on both hosts, with identical values.
3. If using the AES traffic extension, replace `conf/aes.json` with the encrypted AES key expected by `extensions/aes.js`; otherwise do not enable that extension.

Generate the AES key with `lib/crypt.js`:

```
node lib/crypt.js encrypt "<your-secret-key>"
```

WARNING

Never put secrets directly in source control. Any string config value may instead use `env:VARIABLE_NAME`; Reflection resolves it from the process environment at startup.

Example `crypt.json` pattern using environment variables:

```
{
  "key": "env:REFLECTION_CRYPT_KEY",
  "heartbeat_key": "env:REFLECTION_HEARTBEAT_KEY",
  "fips_mode": false,
  "use_checksum": false
}
```

Set those environment variables in the service manager, not in a world-readable shell file. Both endpoints must receive exactly the same values. If FIPS is enabled on the installation system, then change `fips_mode` flag as `true`. If using the AES extension, change the `use_checksum` flag to `true`.

7 Configure a standard TCP relay

7.1 Mirror host: `conf/mirror.json`

Set the bind addresses and shared control ports. A conservative example:

```
{
  "host": "0.0.0.0",
  "port": 2000,
  "heartbeat_tcp": 2001,
  "keep_alive": 1000,
  "timeout": 300000,
  "firewall_enabled_tcp": false,
  "firewall_enabled_udp": false
}
```

`port` is the Reflection control/data port, not the client-facing service port. The reflector asks the mirror to create the client-facing listener dynamically. `host` should be a specific public or private address where possible; use `::` only when intentionally accepting IPv6.

7.2 Reflector host: `conf/reflect.json`

For an origin service at 10.0.0.25:8080, published as mirror port 1201:

```
{
  "reflected_host": "10.0.0.25",
  "reflected_port": 8080,
  "mirror_host": "<mirror_ip>",
  "mirror_port": 2000,
  "mirror_heartbeat_tcp": 2001,
  "mirror_service_host": "0.0.0.0",
  "mirror_service_port": 1201,
  "mirror_force_listener": true,
  "pool_size": 5,
  "mirror_connect_timeout": 20000,
  "heartbeat_check_interval": 60000,
  "dhcp_check": true,
  "logfile": "reflector_tcp.log.ndjson"
}
```

`reflected_*` identifies the origin from the reflector's network. `mirror_*` identifies the mirror control endpoint. `mirror_service_*` defines the public listener created on the mirror. `pool_size` is the number of idle TCP channels; it also caps the number of simultaneous TCP client sessions for this mapping. Size it for expected concurrency plus a small buffer.

For IPv4/IPv6 UDP, also set `mirror_hostudp4`, `mirror_hostudp6`, `mirror_service_host_udp4`, `mirror_service_host_udp6`, and `ipv6enabled` appropriately. Disable IPv6 (false) unless both hosts and firewall rules support it.

7.3 Start order and test

Start the mirror first, then the reflector:

```
cd /opt/reflection
node --env-file=.env mirror_tcp.js

cd /opt/reflection
node --env-file=.env reflect_tcp.js
```

NOTE

Do not regard an open TCP port alone as proof of success: confirm an actual application response.

```
curl -v http://<mirror_ip>:1201/
```

8 TCP, UDP, and multi-service launch commands

Table 2. Launch commands for each deployment mode.

Use case	Mirror command	Reflector command
TCP	<code>node mirror_tcp.js</code>	<code>node reflect_tcp.js</code>
UDP	<code>node mirror_udp.js</code>	<code>node reflect_udp.js</code>
Many TCP mappings	<code>node mirror_tcp.js</code>	<code>node massreflect_tcp.js</code>

8.1 Multiple TCP mappings

Use `conf/massreflect.json` when one reflector publishes multiple origins or multiple reflectors use the same mirror.

Each `reflected_hosts` string follows this syntax:

```
"origin-host:origin-port->published-bind-address:published-port"
```

NOTE

Can leave the `Mirror_hostsudpv6` blank if using ipv4.

```
[
  {
    "reflected_hosts": ["216.116.72.174:80->[::]:1201", "216.116.72.174:443->[::]:1202"],
    "mirror_host": "<mirror_ip>:<mirror_port>",
    "mirror_hostsudpv4": "<mirror_ipv4>:<mirror_port>",
    "Mirror_hostsudpv6": "<mirror_ipv6>:<mirror_port>",
    "mirror_heartbeat_tcp": 2001,
    "mirror_heartbeat_udp": 1002
  }
]
```



SUPPORT

Before contacting support, collect: the document revision shown on the cover; the output of `node --version`; the mirror and reflector log files under `logs/*` for the affected time window; and the exact `conf/*` .json values in use, with every key, `heartbeat_key`, and `aes.json` key field redacted before sending them to anyone.

EMAIL

support@tekmonks.com

HOURS

9.00 am to 9.00 pm IST (Monday - Friday)

REFLECTION · DEPLOYMENT & INSTALLATION GUIDE

Part I · Version 1.0

TEKMONKS

© Tekmonks. All rights reserved.